605.486 – Android Development

Module 24
HTML Applications

HTML-Based Applications

- Web Applications
  - Traditional Web Application
  - AJAX

Web Application: Pros

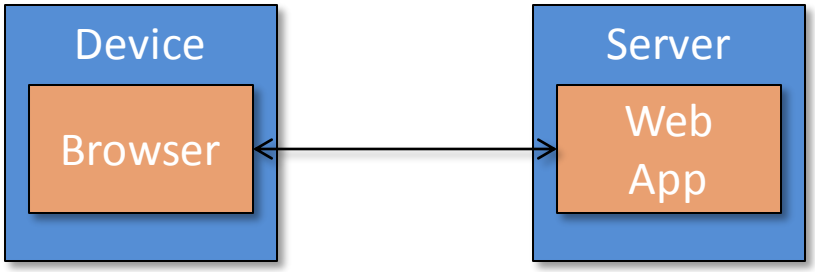
- May be able to develop once, run on any device
  - OS doesn't matter (desktop, Android, iOS)
  - OS Version doesn't matter
- Resources kept somewhere else
- No install needed
- No need to update apps on phone
  - Data kept on server
    - Sometimes necessary (employee benefits, shopping, banking…)
    - Sometimes useful (multiple-device sync, backups…)
    - May be simpler than sending data from an application

Web Applications: Cons

- Requires network connection
- Some sites don't render or act properly
  - Need to consider that they might run on phone, tablet, laptop, desktop…
- Less control over user experience
  - User can navigate away
- User experience doesn't look/feel native
- Hardware access limited/difficult
- Retrieves user interface from server
  - Eats user data
  - Same interface retrieved repeatedly
  - May increase application start time

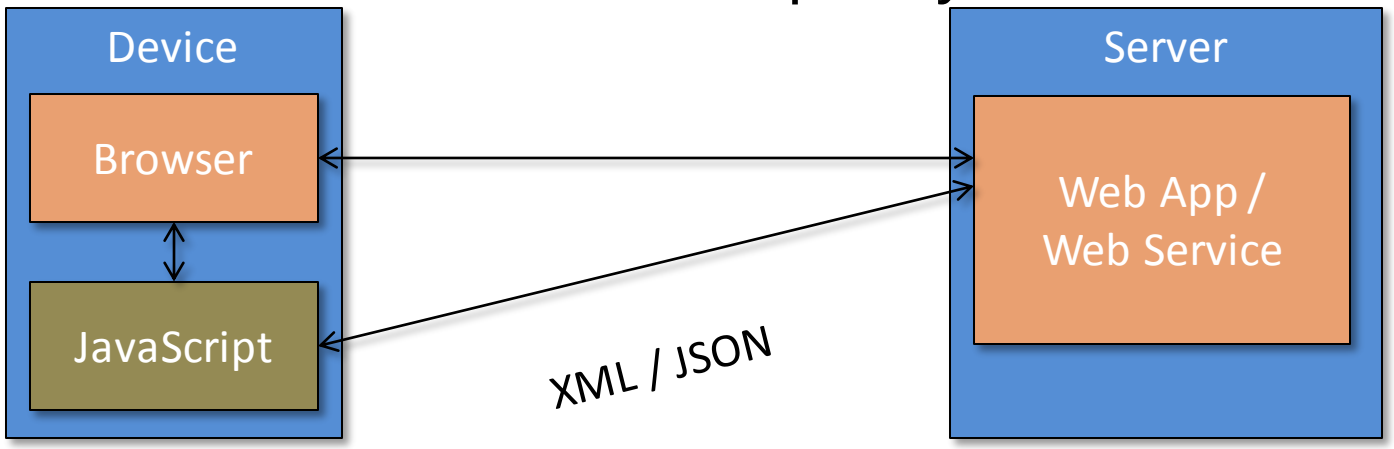
Traditional Web Application

- Aka “thin client”
- All processing on server; Separate request per page
- Client just renders and sends request
- Entire page replaced for every request
  - Likely lots of duplicate data
- Pages very static; little user interaction
- Communication to server infrequent



AJAX Web Application

- "Asynchronous JavaScript and XML"
- Client uses JavaScript to interact with user
- Main data manipulation on server
- Requests sent to server; client updates DOM
- Some interaction can be completely handled on client

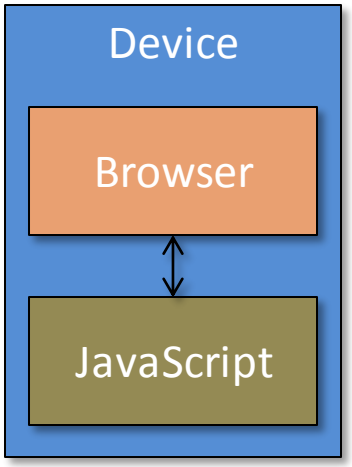


Page Rendering

- Can use any browser on device
  - User enters URL to start application
  - Easy to spoof/phish
- Can use application with embedded WebKit
  - Startup as normal application
  - Always goes to our URL

Local HTML-Based Application

- Uses HTML5/JavaScript
- JavaScript can access device-specific features
  - Separate app per device
    - Glue between JS and phone-native code
  - Frameworks like PhoneGap
    - Might not be able to access some device features



Example 1 - Simple Web Application

- WebView
- Load URL to display page
- Requires INTERNET permission
- Cannot zoom
- Clicking links send Intent – handled by normal browser!
  - We've lost control!

Example 2 - Enabling more controls

- Zoom controls
- Wide viewport
- Overview mode

Example 3 - Keeping links inside app

- Control all links
  - setWebViewClient(new WebViewClient())

- Can control specific links
  - Override shouldOverrideUrlLoading

Example 4 - Pressing "Back"

- "Back" acts on *Activity*, not browser content
  - Really not ideal here…

```
@Override
public void onBackPressed() {
    if (webView.canGoBack())
        webView.goBack();
    else
        super.onBackPressed();
}
```

Example 5 - JavaScript Glue

- Create class that JavaScript will talk to
  - Can be any class, named anything (may be useful to pass a Context)
  - Methods will be available in JavaScript

```
webView.getSettings()
        .setJavaScriptEnabled(true);

webView.addJavascriptInterface(
     new JavaScriptGlue(this),  // glue object
     "Android");                // name in JS
```

- Note: Ensure that *only* your HTML can be displayed!
  - Make sure all links are internal to your app!!!
- Note: If targeting API 17 or later, methods must be annotated with **@JavascriptInterface**

Example 6 - Separate JS Method

```
<input type="button" value="Say hello2"
       onClick="toast('Hello Android!')" />

<script type="text/javascript">
  Android.log("Defining toast function");
  function toast(toast) {
    Android.toast(toast);
  }
</script>
```

Example 7 - JavaScript Console (1)

- webView.setWebChromeClient(**new WebChromeClient() {…}**)
- Override onConsoleMessage
- Can override onJsAlert
  - ICS and later – WebChromeClient automatically displays dialog

Example 7 - JavaScript Console (2)

- Call console and alert in JavaScript
- TEST ERROR: change first HTML button to call Android.xxx

Example 8 - Call JS from Activity

- Write JavaScript function (setMessage)

- Call

```
webView.loadUrl(
        "javascript:setMessage('xxxxx')");
```

jQuery/jQuery Mobile

- Simplifies DOM manipulation in JavaScript
- Runs client-side
- Many custom widgets
- Handles javascript events
- Easily send AJAX requests
- Cross-browser compatibility very good
- jQuery mobile

jQuery Basics (1)

- See http://learn.jquery.com

- $(function() {…})
  - Runs the function when document is ready – good for init

- Selections
  - $("li") – returns all <li> elements
  - $("#foo") – returns the element with id="foo"
  - $(".fee") – returns all elements with class="fee"

- $("a").attr("href") – returns the href attribute of the first <a>
- $("a").attr("href", "http://javadude.com") – sets the href of the first <a>

jQuery Basics (2)

- $(…).html("Hello") changes the content of the first selected element (.html() will get the current content)
- $(…).val("x") – sets the value of a form element (.val() gets it)
- $(…).click(function() {…}) – runs the function when the selected element is clicked
  - Passes an "event" object – you can get location, type, element…
- $(…).hide() – hides the element (it's still in the DOM)
- $(…).show() – shows the element again

Example 9 - Initial jQuery pages (1)

- Include in <head>
  - Stylesheet for jQuery mobile
  - JavaScript for jQuery and jQuery mobile

- Basic list
  - Note the data-xxxx attributes

Example 9 - Initial jQuery pages (2)

- Changing HTML content on button click
- Replacing content with content from a website
- Reading JavaScript objects from a JSON file

Example 10 - HTML5 Local Databases

- You can work with local databases on the phone directly in html 5

- In Activity

```
webView.getSettings().setDatabaseEnabled(true);
String dbPath =  getApplicationContext()
  .getDir("database", Context.MODE_PRIVATE).getPath();
webView.getSettings().setDatabasePath(dbPath);
```

- In JavaScript

```
var db = window.openDatabase(…)
db.transaction(function(tx) {
  tx.executeSql('…');
});
```

Cross-Platform Development (1)

- Many frameworks exist (PhoneGap, Corona, RhoMobile…)

- Pros
  - Develop once, run everywhere
  - Can leverage web developers (most frameworks use HTML 5/JS)
  - Only need to learn one API

Cross-Platform Development (2)

- Cons
  - It just doesn't look and feel like an app!
  - Some platform features may not be available
  - May be a little slower
  - Most are commercial
    - fees vary greatly